

Spike: 5**Title:** Tactical Steering - Hide**Author:** Pattarapol Tantechasa, 103883220**Goals / deliverables:****Expected Outcomes (Baseline Task)**

1. A working simulation featuring one wandering hunter, one prey, and multiple circular obstacles.
2. Agents must possess distinctly different visual appearances and physics properties (mass, speed limits).
3. The simulation must dynamically render geometric indicators ("x") for all potential hiding spots relative to the hunter's current position.
4. The prey agent must autonomously select the optimal hiding location and successfully steer towards it without intersecting the obstacle.
5. A completed written Spike Report detailing the implementation.

Expected Outcomes (from extensions)

1. Working code demonstrating robust obstacle avoidance that prevents agents from colliding with the environment while fleeing/hunting.
2. Implementation of complex tactical analysis where a prey agent successfully calculates and navigates safe paths that avoid line-of-sight during transit or balances multi-hunter threat vectors.
3. A functional game loop where a hunter actively pursues and consumes prey, demonstrating state transitions between wandering and chasing.

Technologies, Tools, and Resources used:

- Claude: brainstorm ideas, explain code and debugs.
- Visual Studio Code IDE
- Conda: Python environment manager

Tasks undertaken:

1. Download and Install **Visual Studio Code**
2. Install **Python** and **Pyglet** on your Machine
3. Open the main project downloaded
4. Open terminal and run cd command : **cd Portfolio-Tasks/Portfolio-Task-13-Spike-Tactical-Steering-Hide**
5. For baseline version : **cd tactical-steering-baseline**
6. For extended version : **cd tactical-steering-extensions**
7. Run : **python main.py**

Design Phase: Tactical Steering (Hide!)

Overview

The goal of this spike was to build a tactical evaluation system where a prey agent can intelligently hide from a hunter by using geometric analysis to identify valid hiding spots behind static obstacles. Unlike reactive steering behaviours (seek, flee, arrive), hiding requires the prey to reason about its environment — evaluating each obstacle relative to the hunter's position and selecting the best concealment point.

I used the existing steering codebase from Task 12 (wander and arrive already implemented) as the base and built the hide system on top without removing existing behaviours.

Environment Setup

I added a static Obstacle class to the World module. Each obstacle holds a position vector and a radius, and renders as a dark green filled circle via Pyglet.

```
class Obstacle:
    def __init__(self, x, y, radius):
        self.pos = Vector2D(x, y)
        self.radius = radius
        self.shape = pyglet.shapes.Circle(
            x, y, radius,
            color=COLOUR_NAMES['DARK_GREEN'],
            batch=window.get_batch("main")
        )
```

I placed five obstacles at fixed positions spread across the window so the prey always has multiple hiding options regardless of where the hunter is. Fixed placement was chosen to keep the simulation deterministic and easy to demonstrate.

Agent Generation and Visual Distinction

I created two agent roles, hunter and prey from the same Agent class but with different constructor parameters to give them distinct visual and physical identities. I also exposed *wander_jitter* as a constructor parameter so each agent type can have independently tuned turning behaviour.

Property	Hunter	Prey
Colour	Red triangle	Green triangle
Mass	1.0	0.8
Max Speed	150 px/s	300 px/s
Max Force	3000	4000
Vehicle Size	1.8x	1.0x
Wander Jitter	20.0	15.0 (default)

The hunter is larger and visually distinct. The prey is lighter with higher force, giving it a tighter turn radius so it responds quickly to changing hide spot targets. Making *wander_jitter* a parameter was necessary because a high-speed heavy agent needs a stronger jitter signal to produce visually natural turns. The same value that looks smooth on the prey made the hunter move in very wide.

Hunter Behaviour — Wander

I set the hunter to use the existing wander steering behaviour from Task 12. A projected jitter circle drifts each frame, creating smooth and unpredictable movement across the environment without any awareness of the prey.

```
def wander(self, delta):
    jitter = self.wander_jitter * delta
    self.wander_target += Vector2D(uniform(-1, 1) * jitter, uniform(-1, 1)
    * jitter)
    self.wander_target.normalise()
    self.wander_target *= self.wander_radius
    target_local = self.wander_target + Vector2D(self.wander_dist, 0)
    world_target = self.world.transform_point(target_local, self.pos,
    self.heading, self.side)
    return self.seek(world_target)
```

In the baseline the hunter never targets the prey deliberately — keeping the focus on the prey's tactical evaluation.

Tactical Analysis — Calculating Hide Spots

For each obstacle, I draw an imaginary ray from the hunter through the obstacle centre and extend it by the obstacle's radius plus a buffer. This projected point is the hiding spot — guaranteed to be on the far side of the obstacle from the hunter. The calculation runs every frame inside `World._calculate_hide_spots()` so spots update in real time as the hunter moves.

```
to_obs = obs.pos - hunter.pos
to_obs.normalise()
spot = obs.pos + to_obs * (obs.radius + BUFFER)
```

This produces one candidate hiding spot per obstacle per frame.

Visualising the Data

I rendered a small white "x" at each calculated hiding spot using two crossing `pyglet.shapes.Line` primitives per obstacle. The marker for the currently selected best spot is drawn in yellow. This visualisation confirmed the geometry was correct during development. I could immediately see the markers shift to the opposite side of each obstacle as the hunter moved.

```
M = 8 # half-size of the x marker in pixels
self._marker_half = M
self.hide_markers = []
for _ in self.obstacles:
    l1 = pyglet.shapes.Line(0, 0, 1, 1,
                            color=COLOUR_NAMES['WHITE'],
                            batch=window.get_batch("main"))
    l2 = pyglet.shapes.Line(0, 0, 1, 1,
                            color=COLOUR_NAMES['WHITE'],
                            batch=window.get_batch("main"))
    self.hide_markers.append((l1, l2))
```

Select and Seek — Prey Behaviour

Each frame the world computes all hiding spots and stores them in `world.hide_spots`. It then selects the best one and stores it in `world.best_hide_spot`. The prey's `hide()` method reads this value and uses `arrive` to steer toward it.

```
# In World._calculate_hide_spots():
best_idx = min(range(len(spots)),
                key=lambda i: self.prey.pos.distance(spots[i]))
self.best_hide_spot = spots[best_idx]

# In Agent.hide():
def hide(self):
    spot = self.world.best_hide_spot
    if spot is None:
        return Vector2D()
    return self.arrive(spot, 'normal')
```

Using `arrive` rather than `seek` prevents the prey from overshooting and oscillating when it reaches the hiding position. The hide spot is always projected outside the obstacle boundary (radius + buffer), so the prey never targets a point inside an obstacle.

Extension 1: Obstacle Avoidance

The baseline hide spot destination is always outside an obstacle, but the travel path to it could clip through an obstacle. To fix this I added a proximity-based repulsion force that runs for every agent on every frame, blended on top of whatever primary steering is active.

```
def obstacle_avoidance(self):
    DANGER_BUFFER = 30.0
    REPULSION_STRENGTH = 15.0
    steer = Vector2D()
    for obs in self.world.obstacles:
        to_agent = self.pos - obs.pos
        dist = to_agent.length()
        danger_radius = obs.radius + DANGER_BUFFER
        if 0.001 < dist < danger_radius:
            overlap = danger_radius - dist
            steer += to_agent.normalise() * (overlap * REPULSION_STRENGTH)
            # Hard correction: eject agent if it fully overlaps
            if dist < obs.radius:
                self.pos = obs.pos + to_agent.normalise() * (obs.radius +
1)
    return steer
```

The repulsion force scales with how deep inside the danger zone the agent is — the closer it gets, the stronger the push. A hard position correction also ejects agents immediately if they somehow overlap a circle, preventing tunnelling at high speed. The outcome is both agents curving naturally around obstacle edges during transit rather than clipping through them.

Extension 2: Multiple Hunters

I extended the simulation to support three independent hunters (red, orange, and pink triangles), each wandering on its own wander state. The world stores them in `world.hunters` instead of a single `world.hunter`.

The prey's hide spot calculation identifies the nearest hunter as the primary threat each frame, since that hunter poses the most immediate danger.

```
nearest = min(self.hunters, key=lambda h: h.pos.distance(self.prey.pos))
```

Hide spots are then calculated relative to that hunter, and the yellow X updates to reflect the most dangerous threat at that moment. Each hunter independently checks its own line of sight and can switch between wander and chase states — so multiple hunters can be chasing simultaneously from different directions.

The visual outcome is noticeably more complex: the prey may have one hunter chasing it directly while others are wandering nearby, and the hide spot selection adapts to whichever is closest at each frame.

Extension 3: Danger-Aware Path Scoring

In Extensions 1 and 2 the prey always chose the hiding spot closest to itself. This is naive — a nearby spot may require the prey to travel directly across a hunter's path to reach it.

I added a `path_exposure()` method on the prey that projects each hunter position onto the straight-line segment from the prey to a candidate spot. If any hunter falls within a threat radius of that path, exposure accumulates. The final score for each candidate combines distance and exposure:

```
def path_exposure(self, target_spot):
    THREAT_RADIUS = 150.0
    exposure = 0.0
    to_target = target_spot - self.pos
    seg_len_sq = to_target.dot(to_target)
    if seg_len_sq < 0.001:
        return 0.0
    for hunter in self.world.hunters:
        to_hunter = hunter.pos - self.pos
        t = max(0.0, min(1.0, to_hunter.dot(to_target) / seg_len_sq))
        closest = self.pos + to_target * t
        dist = hunter.pos.distance(closest)
        if dist < THREAT_RADIUS:
            exposure += (THREAT_RADIUS - dist)
    return exposure

# Selection score used in World._calculate_hide_spots():
score = dist_to_spot + path_exposure * 2.0
```

The result: when a hunter sits between the prey and a nearby obstacle, the yellow X jumps to a farther obstacle whose approach path is clear. The prey accepts a longer route in exchange for a safer one — a meaningfully more intelligent decision than pure proximity.

Extension 4: Hunter Chase, Eat, and Respawn

I gave each hunter a two-state behaviour: wander when the prey is out of sight, chase when it is visible.

Line-of-sight check: Each frame I cast a ray from the hunter to the prey and test it against every obstacle circle using the quadratic segment-circle intersection formula. If any obstacle occludes the ray, the hunter cannot see the prey.

```
def _ray_intersects_circle(start, end, center, radius):
    d = end - start
    f = start - center
    a = d.dot(d)
    b = 2.0 * f.dot(d)
    c = f.dot(f) - radius * radius
    discriminant = b * b - 4.0 * a * c
    if discriminant < 0:
        return False
    sq = discriminant ** 0.5
    t1 = (-b - sq) / (2.0 * a)
    t2 = (-b + sq) / (2.0 * a)
    return (0.0 <= t1 <= 1.0) or (0.0 <= t2 <= 1.0)
```

Pursuit: When a hunter has line of sight it switches to chase mode, which uses a pursuit behaviour that predicts the prey's future position based on its current velocity and steers toward it.

```
def pursuit(self, evader):
    to_evader = evader.pos - self.pos
    look_ahead = to_evader.length() / (self.max_speed + evader.speed())
    future_pos = evader.pos + evader.vel * look_ahead
    return self.seek(future_pos)
```

An orange line is drawn from each chasing hunter to the prey while line of sight is active, making the state switch clearly visible.

Catch and respawn: If any hunter closes within 25 pixels of the prey, the prey teleports to a random position, its velocity resets to zero, and a screen counter increments.

```
def _check_catch(self):
    CATCH_RADIUS = 25.0
    for hunter in self.hunters:
        if hunter.pos.distance(self.prey.pos) < CATCH_RADIUS:
            self.score += 1
            self.score_label.text = f'Caught: {self.score}'
            self.prey.pos.x = randrange(self.cx)
            self.prey.pos.y = randrange(self.cy)
            self.prey.vel.x = 0.0
            self.prey.vel.y = 0.0
            break
```

This creates a self-sustaining game loop: hunters wander, spot prey, pursue, catch, then return to wandering. The prey continuously evaluates hiding spots and reacts to the changing threat landscape, producing emergent predator-prey dynamics.

Conclusion

I built a working tactical evaluation system that allows a prey agent to reason geometrically about its environment rather than simply reacting to the nearest threat. The hide spot calculation — projecting a point through each obstacle away from the hunter is mathematically simple but produces believable, purposeful movement.

The baseline demonstrated that even a single-criterion evaluation (closest hiding spot) produces convincing avoidance behaviour as long as hide spots are recalculated dynamically each frame. The extensions progressively raised the sophistication: obstacle avoidance eliminated physical clipping during transit, multiple hunters forced the prey to identify the most immediate threat, danger-aware path scoring penalised exposed approach routes, and the full hunt cycle produced a self-sustaining game loop.

The key insight from this spike is that tactical intelligence does not require complex planning. It emerges from evaluating a small set of well-chosen geometric properties (position, line of sight, path exposure) at runtime.